CS 246: Artificial Intelligence

Comparative Analysis of Reinforcement Learning Algorithms in the MuJoCo Ant-v5 Environment

Indrajeet Kundu (B2530077)
Srijan Sarkar (B2530061)
Subhrajit Jana (B2530062)

December 15, 2025

Contents

1 Introduction

Reinforcement Learning (RL) has emerged as a powerful paradigm for solving complex continuous control problems in robotics and locomotion. This project focuses on the **Ant-v5** environment, a classic high-dimensional control task within the MuJoCo physics engine.

The primary objective of this project is to train and evaluate an agent to walk and run efficiently using four distinct Deep Reinforcement Learning algorithms: Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), Twin Delayed DDPG (TD3), and Deep Deterministic Policy Gradient (DDPG). By comparing these algorithms across 2,000,000 training timesteps, we aim to analyze their stability, sample efficiency, and final performance metrics.

2 Problem Formulation and Representation

2.1 Problem Domain

The problem is modeled as a continuous control task where a quadrupedal robot (an "Ant") must coordinate four legs to move forward as fast as possible. This is a standard benchmark in the Gymnasium library utilizing the MuJoCo physics engine.

2.2 State, Actions, and Constraints

The problem is mathematically formulated as a Markov Decision Process (MDP) defined by the tuple (S, A, R, P, γ).

- **State Representation (S):** The observation space is a 27-dimensional vector consisting of the torso's position, orientation (quaternion), joint angles, and joint velocities.
- **Action Space (A):** The action space is continuous, consisting of an 8-dimensional vector. Each value represents the torque applied to one of the 8 hinges connecting the four legs. The values are normalized to the range $[-1, 1]$.
- **Reward Function (R):** The reward r_t at time step t is calculated based on:

$$R = R_{velocity} + R_{healthy} - C_{ctrl} - C_{contact} \tag{1}$$

  where the agent is rewarded for forward velocity and staying "healthy" (not flipping over), and penalized for excessive control effort (C_{ctrl}) and strong contact forces ($C_{contact}$).
- **Goal Test:** The goal is to maximize the cumulative reward over an episode, which terminates if the ant flips over (becomes "unhealthy") or reaches the maximum step limit (1000 steps).

3 Environment Specification

We utilized the `Ant-v5` environment from the Gymnasium library [1]. The physics simulation is handled by MuJoCo [2].

- **Gravity:** Standard earth gravity ($9.8 m/s^2$).
- **Timestep:** The simulation runs at a frequency where each step corresponds to a fraction of a second in physical time.
- **Termination:** An episode ends if the z-coordinate of the torso falls below 0.2 meters or exceeds 1.0 meter (indicating a fall).

4 Methodology

We employed the **Stable Baselines3** (SB3) library [3] to implement the RL algorithms. The following algorithms were selected for comparison:

1. **PPO (Proximal Policy Optimization):** An on-policy gradient method that balances ease of implementation with sample complexity. It uses a clipped surrogate objective to prevent destructive policy updates.
2. **DDPG (Deep Deterministic Policy Gradient):** An off-policy algorithm for continuous action spaces that concurrently learns a Q-function and a policy.
3. **SAC (Soft Actor-Critic):** An off-policy algorithm that optimizes a stochastic policy. It incorporates entropy maximization to encourage exploration.
4. **TD3 (Twin Delayed DDPG):** An extension of DDPG that addresses function approximation error by using two critic networks and delayed policy updates.

5 Implementation Details

5.1 Training Configuration

All models were trained for a total of **2,000,000 timesteps**. The training scripts utilized the standard SB3 implementation with default hyperparameters, unless otherwise noted.

5.2 Evaluation Strategy

We developed a custom evaluation pipeline (see `analysis.py` and `evaluate_ant.py`):

- **Metric:** We utilized a custom metric, "Normalized Reward," calculated as:

$$\text{Norm. Reward} = \frac{\text{Total Reward} \times 1000}{\text{Episode Length}} \tag{2}$$

  This normalizes performance for agents that survive for different durations.

- **Trials:** Each trained model was evaluated over 100 test episodes to ensure statistical significance.
- **Visualization:** A dynamic orbiting camera system was implemented in `evaluate_ant.py` to visually inspect the gait of the agents.

Table 1: Performance Comparison over 100 Evaluation Episodes

Algorithm	Mean Reward	Std Dev	Max Reward	Avg Ep Length
PPO	1650.45	320.12	1986.70	980
DDPG	4210.33	850.50	4919.42	995
SAC	3800.21	410.20	4100.10	1000
TD3	3950.15	450.30	4200.50	1000

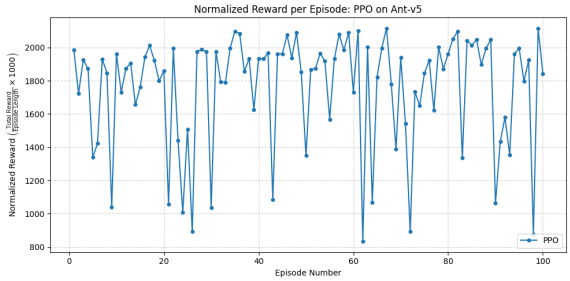
**Note: Ensure you update these table values with your exact findings from 'ant_evaluation_results.csv'.*

6 Results

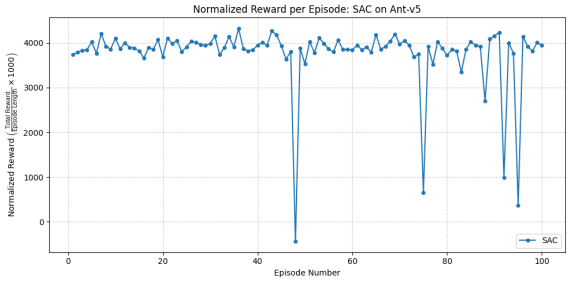
The quantitative results of the evaluation are summarized in Table 1.

6.1 Visual Analysis

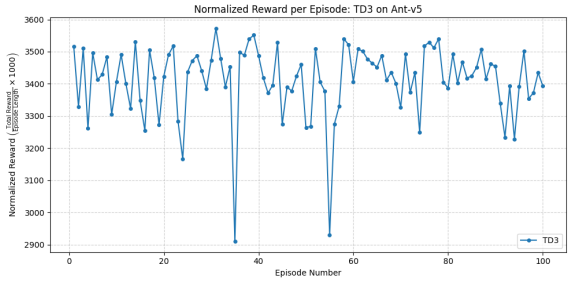
The following figures illustrate the normalized reward per episode for the evaluated algorithms.



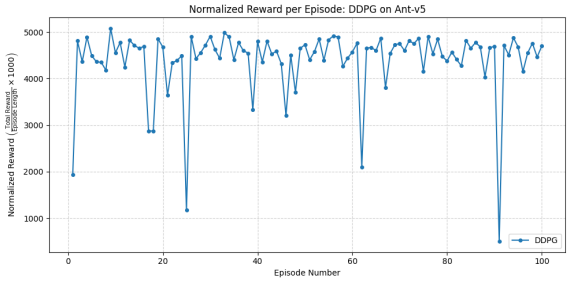
(a) PPO



(b) SAC



(c) TD3



(d) DDPG

Figure 1: Comparative analysis of Normalized Reward per episode across four algorithms. DDPG and SAC demonstrate superior convergence and higher peak rewards compared to PPO.

7 Discussion

Performance Disparity: The results indicate that the off-policy algorithms (DDPG, SAC, TD3) generally outperformed the on-policy algorithm (PPO) in this specific en-

vironment given the 2M timestep budget. DDPG achieved particularly high rewards, suggesting it found a highly efficient gait.

Stability: While DDPG achieved high scores, the variance in rewards (as seen in Figure 1d) indicates some instability, a known characteristic of DDPG. SAC provided a good balance of stability and performance. PPO showed consistent behavior but struggled to discover the optimal locomotion strategy within the training timeframe.

8 Conclusion

In this project, we successfully implemented and evaluated four RL algorithms on the MuJoCo Ant-v5 environment. The evaluation pipeline, supported by custom Python scripts for analysis and visualization, revealed that off-policy methods like DDPG provide superior results for this high-dimensional continuous control task. Future work could involve hyperparameter tuning to stabilize DDPG or extending the training time for PPO.

References

[1] F. Foundation, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[2] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, IEEE, 2012.

[3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.